EECS 3311

Software Design Fall 2026

Sections Z

Deliverable 3

PART 1: Manually Written JUnit Tests

We created a dedicated test folder and wrote at least 10 JUnit test cases per non-GUI class. Each test was designed to target specific logic paths, edge cases, and exception handling. We used IntelliJ's coverage tool to verify that our manually written tests achieved an average code coverage of 97% and mutation coverage of 69%, with a test strength of 72%.

Pit Test Coverage Report for Human tests

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
17	97%	203/210	69%	64/93	72%	64/89

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.yorku.booking	2	100%	44/44	25%	6/24	25%	6/24
com.yorku.command	3	100%	23/23	67%	2/3	67%	2/3
com.yorku.coordinator	2	94%	82/87	78%	32/41	86%	32/37
com.yorku.observer	2	100%	12/12	67%	2/3	67%	2/3
com.yorku.users	8	95%	42/44	100%	22/22	100%	22/22

Report generated by PIT 1.23.0

Enhanced functionality available at arcmutate.com

PART 2: Randoop Generated Tests

We followed the provided tutorial to run Randoop on our compiled project classes. The generated tests were stored in a separate Randoop folder. Some tests were removed due to runtime failures or irrelevance. The remaining tests were compiled and executed successfully. Randoop achieved 80% line coverage, 40% mutation coverage, and 58% test strength. While it explored many paths, it missed domain specific logic and produced some redundant tests. Some tests were removed due to runtime failures or irrelevance. The remaining tests were

compiled and executed successfully. Randoop achieved lower coverage than manual tests but still contributed to exploring unexpected paths.

Pit Test Coverage Report for randoop

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
17	80%	169/210	40%	37/93	58%	37/64

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.yorku.booking	2	64%	28/44	17%	4/24	29%	4/14
com.yorku.command	3	100%	23/23	67%	2/3	67%	2/3
com.yorku.coordinator	2	80%	70/87	34%	14/41	52%	14/27
com.yorku.observer	2	100%	12/12	67%	2/3	67%	2/3
com.yorku.users	8	82%	36/44	68%	15/22	88%	15/17

Report generated by PIT 1.23.0

Enhanced functionality available at arcmutate.com

PART 3: AI Assisted Tests

B. Coverage Report

We used PIT to measure mutation coverage for the AI-generated tests. The overall mutation coverage was 72%, line coverage was 94%, and test strength was 78%.

Pit Test Coverage Report for AI tests

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
17	94%	197/210	72%	67/93	78%	67/86

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.yorku.booking	2	86%	38/44	38%	9/24	43%	9/21
com.yorku.command	3	100%	23/23	100%	3/3	100%	3/3
com.yorku.coordinator	2	97%	84/87	80%	33/41	85%	33/39
com.yorku.observer	2	100%	12/12	67%	2/3	67%	2/3
com.yorku.users	8	91%	40/44	91%	20/22	100%	20/20

Report generated by PIT 1.23.0

Enhanced functionality available at arcmutate.com

C. AI Assistant Analysis

The AI assistant used was the github copilot. The AI responded to each prompt quickly but always added some unnecessary features to the project. Each test was accepted and the only modification was to add more tests for more coverage. No prompt refinement steps where needed. Our understanding of the classes allowed us to identify gaps in the AI's approach.We prioritized broader scenario coverage. We verified correctness by comparing runtime outputs with expected behavior. No test failed or produced incorrect assertions. No test failed at run time or provided incorrect assertions. Examples of successful test cases that passed and provided meaningful, accurate coverage would be

1. extendReservationUpdatesReservedHours

```java
@Test
void extendReservationUpdatesReservedHours() throws Exception {
    BookingFacade facade = new BookingFacade();
    User user = new Student(email: "student3@yorku.ca", password: "pass", idNumber: "S3");
    Equipment equipment = new Equipment(id: "EQ-3", description: "Scope", location: "Lab C");
    facade.reserveEquipment(user, equipment, hours: 2);

    facade.extendReservation(user, equipment, additionalHours: 4);

    assertTrue(getReservations(facade).get(user).get(equipment) == 6);
}
```

2. testClonePreservesData

```java
@Test
void testClonePreservesData() {
    Equipment eq1 = new Equipment(id: "E1", description: "Drill", location: "Room A");
    Equipment eq2 = eq1.clone();

    assertEquals(expected: "E1", eq2.getId());
    assertEquals(expected: "Drill", eq2.getDescription());
    assertEquals(expected: "Room A", eq2.getLocation());
}
```

PART 4: Comparison Report

Dimension	Manual Tests	AI Generated Tests	Randoop Tests
Code coverage	Highest	Second highest	Second highest
Reason	Most likely best because we intentionally targeted	They systematically explored many paths But may miss very	Excels at brute-force exploration But coverage can

	edge cases and known logic paths and some tests were deleted from randoop since they failed at run time	specific or domain-driven cases	include irrelevant and redundant paths
Mutation score	Second highest	Highest	Lowest
Readability	Best	Second best	Worst
reason	Clear structure,Easy to understand intent	Generally readable, but: Less semantic naming	Poor readability,Random method calls
Usefulness	Highest	Second highest	Low
reason	High practical value: Test real use cases	Good usefulness: Strong at catching edge cases	Low usefulness: Many tests are trivial or irrelevant

Final Reflection

AI-assisted testing helped accelerate test creation and filled gaps in our manual suite. Its strengths include speed and baseline coverage, but it lacks domain understanding and often misses edge cases. Manual testing remains essential for precision and reliability. Randoop is useful for brute-force exploration but requires heavy curation. Overall, combining all three approaches helped us achieve robust coverage and improved confidence in our system's reliability.